si, nell'implementazione concreta usiamo un parser YAML affidabile (che potrebbe essere Python + PyYAML, e vorrei anche una valutazione con Perl + Template::toolkit) ed usiamo template Compose per servizio (services/postgres/templates/compose.yaml.j2) come suggerito.

Infine, in base alla scelta della soluzione Python oppure Perl, vediamo come integrare in docker-dev.sh

La scelta più pragmatica è **Python + PyYAML + Jinja2**, soprattutto se `docker-dev` deve restare manutenibile e portabile tra i tuoi ambienti Linux/WSL2. **Perl + YAML + Template::Toolkit** è perfettamente valido e molto elegante per il templating, ma introduce una dipendenza CPAN e una seconda "familia" di strumenti rispetto a Python, che probabilmente usi già per pipeline e data processing. PyYAML espone `safe_load` per YAML non fidato, mentre Template::Toolkit è un motore di templating testuale generale con variabili, include e process. [1] [2] [3]

Valutazione delle due soluzioni

Aspetto	Python + PyYAML + Jinja2	Perl + YAML + Template::Toolkit
Parser YAML	`PyYAML` / `ruamel.yaml`; usa sempre `safe_load` per input non fidato	`YAML::XS` oppure `YAML::Tiny`; `YAML::XS` è robusto e veloce
Templating	Jinja2: sandboxing opzionale, filtri, ereditarietà, autoescape per HTML ma utile anche per YAML	Template::Toolkit: molto espressivo, `INCLUDE`, `PROCESS`, macro, plugin
Dipendenze	`python3`, `PyYAML`, `Jinja2`; facili con venv o pacchetti di distro	`perl`, `YAML::XS`, `Template`; spesso richiede CPAN/cpanm
Integrazione con Bash	Ottima tramite un unico entrypoint `docker-dev-services`	Ottima, ma richiede attenzione a encoding, quoting e versioni modulo
Manutenibilità	Alta se il resto degli strumenti è Python	Alta se il tuo ecosistema è già Perl/Raku-oriented
Rischio principale	Dipendenze Python e virtualenv	Dipendenze CPAN e differenze tra installazioni Perl
Ideale per	Generazione deterministica di YAML da manifest	Template complessi, riuso avanzato, ecosistema Perl

Per `docker-dev` sceglierei **Python**: un solo interprete può fare parsing, validazione, rendering Jinja2, hash di configurazione e emissione del file finale. `yaml.safe_load` è raccomandato per YAML proveniente da fonti non completamente fidate perché limita il caricamento a tipi standard.[4] [1]

Architettura comune

Indipendentemente dal linguaggio, separa tre responsabilità:

```
docker-dev.sh
  └─ 40.services.sh / docker-dev-services
       ├─ risolve --with in tipi di servizio
       ├─ legge services/<type>/service.yaml
       ├─ valida parametri CLI
       ├─ renderizza services/<type>/templates/compose.yaml.j2
       ├─ unisce i frammenti in compose.services.yaml
       └─ emette l'elenco profili
```

Il file generato viene passato a `docker compose` come ultimo `-f`, dopo il Compose base e l'eventuale configurazione utente. Compose merge i file nell'ordine fornito.[5]

Soluzione Python

Dipendenze

```
python3 -m venv .venv
.venv/bin/pip install PyYAML Jinja2
```

In alternativa, su Debian/Ubuntu puoi usare i pacchetti di sistema:

```
sudo apt install python3-yaml python3-jinja2
```

Struttura

```
docker-dev/
├── docker-dev.sh
├── bin/
│   └── docker-dev-services.py
└── services/
    └── postgres/
        ├── service.yaml
        └── templates/
            ├── compose.yaml.j2
            └── Dockerfile
```

services/postgres/service.yaml

```
name: postgres
image: dev.postgres
default_tag: "16"
profiles:
  - db
container_port: 5432
default_host_port: 15432
data_volume:
  name: "data.dev.postgres.{project}"
  mount: /var/lib/postgresql/data
environment:
  POSTGRES_USER: app
  POSTGRES_PASSWORD: app
  POSTGRES_DB: app
connection_env:
  DATABASE_URL: "postgres://app:app@{service}:5432/app"
healthcheck:
  test: ["CMD-SHELL", "pg_isready -U app -d app"]
  interval: 5s
  timeout: 3s
  retries: 10
```

services/postgres/templates/compose.yaml.j2

```
services:
  {{ logical_name }}:
    image: {{ image }}
    profiles:
      - {{ profile }}
    restart: unless-stopped
{% if host_port %}
    ports:
      - "127.0.0.1:{{ host_port }}:{{ container_port }}"
{% endif %}
{% if data_volume %}
    volumes:
      - {{ data_volume }}:{{ data_mount }}
{% endif %}
    environment:
{% for key, value in environment.items() %}
      {{ key }}: {{ value }}
{% endfor %}
    healthcheck:
      test: {{ healthcheck_test }}
      interval: {{ healthcheck_interval }}
      timeout: {{ healthcheck_timeout }}
      retries: {{ healthcheck_retries }}
```

bin/docker-dev-services.py

```
#!/usr/bin/env python3
from __future__ import annotations

import argparse
import hashlib
import json
import os
import subprocess
import sys
from pathlib import Path
from typing import Any

import yaml
from jinja2 import Environment, FileSystemLoader, StrictUndefined

ALIASES = {
    "db": "postgres",
    "cache": "redis",
    "mail": "mailpit",
    "object": "minio",
}


def fail(message: str) -> None:
    print(f"docker-dev services: {message}", file=sys.stderr)
    raise SystemExit(1)


def parse_with(value: str | None) -> list[tuple[str, dict[str, str]]]:
    if not value:
        return []

    requested: list[tuple[str, dict[str, str]]] = []

    for raw in value.split(","):
        raw = raw.strip()
        if not raw:
            continue

        logical_name, separator, raw_params = raw.partition(":")
        params: dict[str, str] = {}

        if separator:
            for pair in raw_params.split(","):
                if not pair:
                    continue
                key, equal, param_value = pair.partition("=")
                if not equal:
                    fail(f"invalid service parameter: {pair}")
                params[key] = param_value

        requested.append((logical_name, params))

    return requested
```

```
def load_manifest(services_root: Path, service_type: str) -> dict[str, Any]:
    manifest_path = services_root / service_type / "service.yaml"

    if not manifest_path.is_file():
        fail(f"unknown service type: {service_type} ({manifest_path} not found)")

    try:
        with manifest_path.open(encoding="utf-8") as stream:
            manifest = yaml.safe_load(stream)
    except yaml.YAMLError as exc:
        fail(f"invalid YAML in {manifest_path}: {exc}")

    if not isinstance(manifest, dict):
        fail(f"service manifest must be a mapping: {manifest_path}")

    for required in ("name", "image", "profiles"):
        if required not in manifest:
            fail(f"missing '{required}' in {manifest_path}")

    return manifest


def render_service(
    services_root: Path,
    service_type: str,
    logical_name: str,
    manifest: dict[str, Any],
    params: dict[str, str],
    project_name: str,
) -> tuple[str, str]:
    tag = params.get("tag", str(manifest.get("default_tag", "latest")))
    image = f"{manifest['image']}:{tag}"

    host_port = params.get("host_port", manifest.get("default_host_port"))
    data_volume = params.get("volume")

    if data_volume is None:
        volume_spec = manifest.get("data_volume") or {}
        volume_name = volume_spec.get("name")
        if volume_name:
            data_volume = volume_name.replace("{project}", project_name)

    template_dir = services_root / service_type / "templates"
    template_path = template_dir / "compose.yaml.j2"

    if not template_path.is_file():
        fail(f"missing Compose template: {template_path}")

    environment = Environment(
        loader=FileSystemLoader(template_dir),
        undefined=StrictUndefined,
        keep_trailing_newline=True,
    )
```

```
    template = environment.get_template("compose.yaml.j2")

    rendered = template.render(
        logical_name=logical_name,
        image=image,
        profile=service_type,
        host_port=host_port,
        container_port=manifest.get("container_port"),
        data_volume=data_volume,
        data_mount=(manifest.get("data_volume") or {}).get("mount"),
        environment=manifest.get("environment", {}),
        healthcheck_test=json.dumps(manifest.get("healthcheck", {}).get("test", []))
        healthcheck_interval=manifest.get("healthcheck", {}).get("interval", "5s"),
        healthcheck_timeout=manifest.get("healthcheck", {}).get("timeout", "3s"),
        healthcheck_retries=manifest.get("healthcheck", {}).get("retries", 10),
    )

    return rendered, image


def build_connection_env(
    manifest: dict[str, Any],
    logical_name: str,
) -> dict[str, str]:
    connection_env = manifest.get("connection_env", {})
    return {
        key: value.replace("{service}", logical_name)
        for key, value in connection_env.items()
    }


def main() -> None:
    parser = argparse.ArgumentParser()
    parser.add_argument("--target", required=True)
    parser.add_argument("--project-name", required=True)
    parser.add_argument("--with", dest="with_services")
    parser.add_argument("--services-file", required=True)
    parser.add_argument("--profiles-file", required=True)
    parser.add_argument("--build-missing", action="store_true")
    parser.add_argument("--dry-run", action="store_true")
    args = parser.parse_args()

    docker_dev_root = Path(
        os.environ.get("DOCKER_DEV_ROOT", Path(__file__).resolve().parent.parent)
    )
    services_root = docker_dev_root / "services"

    services_file = Path(args.services_file).resolve()
    profiles_file = Path(args.profiles_file).resolve()
    services_file.parent.mkdir(parents=True, exist_ok=True)

    requested = parse_with(args.with_services)

    service_blocks: list[str] = []
    dev_environment: dict[str, str] = {}
    dev_depends_on: dict[str, str] = {}
```

```
    profiles: list[str] = []
    volumes: dict[str, dict[str, Any]] = {}

    for logical_name, params in requested:
        service_type = ALIASES.get(logical_name, logical_name)
        manifest = load_manifest(services_root, service_type)

        rendered, image = render_service(
            services_root=services_root,
            service_type=service_type,
            logical_name=logical_name,
            manifest=manifest,
            params=params,
            project_name=args.project_name,
        )

        service_blocks.append(rendered)
        profiles.extend(manifest.get("profiles", []))

        dev_environment.update(
            build_connection_env(manifest, logical_name)
        )

        has_healthcheck = bool(manifest.get("healthcheck"))
        dev_depends_on[logical_name] = (
            "service_healthy" if has_healthcheck else "service_started"
        )

        volume_spec = manifest.get("data_volume") or {}
        volume_name = volume_spec.get("name")
        if volume_name:
            resolved_volume = volume_name.replace("{project}", args.project_name)
            volumes[resolved_volume] = {"external": True}

    if service_blocks:
        dev_block = {
            "dev": {
                "environment": dev_environment,
                "depends_on": dev_depends_on,
            }
        }
        service_blocks.append(yaml.safe_dump(dev_block, sort_keys=False).rstrip())

    if volumes:
        volumes_block = {"volumes": volumes}
        service_blocks.append(yaml.safe_dump(volumes_block, sort_keys=False).rstrip(

    output = "\n\n".join(service_blocks) + "\n"

    if args.dry_run:
        print(output, end="")
        return

    services_file.write_text(output, encoding="utf-8")
    profiles_file.write_text("\n".join(dict.fromkeys(profiles)) + "\n", encoding="ut
```

```
if __name__ == "__main__":
    main()
```

Questo script è un buon punto di partenza, ma nella versione finale sposterei la generazione del blocco `dev` in un template separato, ad esempio `templates/dev-overrides.yaml.j2`, così Python fa solo risoluzione e rendering.

Soluzione Perl

Dipendenze

```
cpanm YAML::XS Template
```

Su Debian, se disponibili:

```
sudo apt install libyaml-libyaml-perl libtemplate-perl
```

Struttura

```
docker-dev/
├── docker-dev.sh
├── bin/
│   └── docker-dev-services.pl
└── services/
    └── postgres/
        ├── service.yaml
        └── templates/
            └── compose.yaml.j2
```

`bin/docker-dev-services.pl`

```
#!/usr/bin/env perl
use v5.36;
use strict;
use warnings;

use Cwd qw(getcwd);
use File::Basename qw(dirname);
use File::Path qw(make_path);
use Getopt::Long qw(GetOptions);
use Template;
use YAML::XS qw(LoadFile);

my %ALIASES = (
    db     => 'postgres',
    cache  => 'redis',
    mail   => 'mailpit',
```

```
    object => 'minio',
);

my (
    $target,         $project_name, $with_services,
    $services_file,  $profiles_file, $build_missing,
    $dry_run,
);

GetOptions(
    'target=s'          => \$target,
    'project-name=s'    => \$project_name,
    'with=s'            => \$with_services,
    'services-file=s'   => \$services_file,
    'profiles-file=s'   => \$profiles_file,
    'build-missing'     => \$build_missing,
    'dry-run'           => \$dry_run,
) or die "Usage: $0 --target TARGET --project-name NAME ...\n";

die "--target is required\n"           unless $target;
die "--project-name is required\n"     unless $project_name;
die "--services-file is required\n"    unless $services_file;
die "--profiles-file is required\n"    unless $profiles_file;

my $docker_dev_root = $ENV{DOCKER_DEV_ROOT}
    // dirname(dirname(__FILE__));
my $services_root = "$docker_dev_root/services";

make_path(dirname($services_file));
make_path(dirname($profiles_file));

my @requested;

if ($with_services) {
    for my $raw (split /,/, $with_services) {
        next unless length $raw;

        my ($logical_name, $params_raw) = split /:/, $raw, 2;
        my %params;

        if (defined $params_raw) {
            for my $pair (split /,/, $params_raw) {
                my ($key, $value) = split /=/, $pair, 2;
                die "invalid service parameter: $pair\n" unless defined $value;
                $params{$key} = $value;
            }
        }

        push @requested, {
            logical_name => $logical_name,
            params       => \%params,
        };
    }
}

my $tt = Template->new({
```

```
    INCLUDE_PATH => [],
    ABSOLUTE     => 1,
    STRICT       => 1,
}) or die Template->error;

my (@service_blocks, @profiles, %dev_environment, %dev_depends_on, %volumes);

for my $request (@requested) {
    my $logical_name = $request->{logical_name};
    my $service_type = $ALIASES{$logical_name} // $logical_name;
    my $manifest_path = "$services_root/$service_type/service.yaml";

    die "unknown service type: $service_type ($manifest_path not found)\n"
        unless -f $manifest_path;

    my $manifest = LoadFile($manifest_path);

    my $params       = $request->{params};
    my $tag          = $params->{tag} // ($manifest->{default_tag} // 'latest');
    my $image        = "$manifest->{image}:$tag";
    my $host_port    = $params->{host_port} // $manifest->{default_host_port};
    my $volume_spec  = $manifest->{data_volume} // {};
    my $data_volume  = $params->{volume};

    if (!defined $data_volume && $volume_spec->{name}) {
        $data_volume = $volume_spec->{name};
        $data_volume =~ s/\{project\}/$project_name/g;
    }

    my $template_path = "$services_root/$service_type/templates/compose.yaml.j2";

    die "missing Compose template: $template_path\n"
        unless -f $template_path;

    my $output;
    $tt->process(
        $template_path,
        {
            logical_name        => $logical_name,
            image               => $image,
            profile             => $service_type,
            host_port           => $host_port,
            container_port      => $manifest->{container_port},
            data_volume         => $data_volume,
            data_mount          => $volume_spec->{mount},
            environment         => $manifest->{environment} // {},
            healthcheck_test    => $manifest->{healthcheck}{test},
            healthcheck_interval => $manifest->{healthcheck}{interval} // '5s',
            healthcheck_timeout => $manifest->{healthcheck}{timeout} // '3s',
            healthcheck_retries => $manifest->{healthcheck}{retries} // 10,
        },
        \$output,
    ) or die $tt->error;

    push @service_blocks, $output;
    push @profiles, @{ $manifest->{profiles} // [] };
```

```
    my $connection_env = $manifest->{connection_env} // {};

    for my $key (keys %$connection_env) {
        my $value = $connection_env->{$key};
        $value =~ s/\{service\}/$logical_name/g;
        $dev_environment{$key} = $value;
    }

    $dev_depends_on{$logical_name} =
        $manifest->{healthcheck} ? 'service_healthy' : 'service_started';

    if ($volume_spec->{name}) {
        my $resolved = $volume_spec->{name};
        $resolved =~ s/\{project\}/$project_name/g;
        $volumes{$resolved} = { external => 1 };
    }
}

if (@service_blocks) {
    my $dev_yaml = YAML::XS::Dump({
        dev => {
            environment => \%dev_environment,
            depends_on  => \%dev_depends_on,
        },
    });
    push @service_blocks, $dev_yaml;
}

if (%volumes) {
    push @service_blocks, YAML::XS::Dump({ volumes => \%volumes });
}

my $output = join "\n\n", @service_blocks;
$output .= "\n";

if ($dry_run) {
    print $output;
    exit 0;
}

open my $services_fh, '>', $services_file
    or die "cannot write $services_file: $!\n";
print {$services_fh} $output;
close $services_fh;

my %unique_profiles = map { $_ => 1 } @profiles;

open my $profiles_fh, '>', $profiles_file
    or die "cannot write $profiles_file: $!\n";
print {$profiles_fh} "$_\n" for sort keys %unique_profiles;
close $profiles_fh;
```

Template::Toolkit supporta variabili Perl di ogni tipo — scalari, liste, hash e oggetti — e direttive come `INCLUDE` e `PROCESS`, quindi è molto adatto a frammenti Compose modulari. [2] [6]

Integrazione in `docker-dev.sh`

Scelta Python

`docker-dev.sh` può invocare direttamente lo script Python:

```
PYTHON_BIN="${PYTHON_BIN:-python3}"
DOCKER_DEV_ROOT="$(cd "$(dirname "${BASH_SOURCE[^0]}")" && pwd)"
SERVICES_SCRIPT="${DOCKER_DEV_ROOT}/bin/docker-dev-services.py"

GENERATED_DIR="${PROJECT_DIR}/.docker-dev/generated"
SERVICES_FILE="${GENERATED_DIR}/compose.services.yaml"
PROFILES_FILE="${GENERATED_DIR}/profiles"

mkdir -p "$GENERATED_DIR"

"$PYTHON_BIN" "$SERVICES_SCRIPT" \
  --target "$TARGET" \
  --project-name "$PROJECT_NAME" \
  --with "$DDEV_WITH" \
  --services-file "$SERVICES_FILE" \
  --profiles-file "$PROFILES_FILE" \
  ${DDEV_BUILD_MISSING:+--build-missing} \
  ${DDEV_DRY_RUN:+--dry-run}
```

Poi costruisci il comando Compose:

```
compose_args=(
  --project-name "$PROJECT_NAME"
  --project-directory "$PROJECT_DIR"
  -f "$PROJECT_DIR/.docker-dev/compose.yaml"
)

if [[ -f "$PROJECT_DIR/.docker-dev/compose.user.yaml" ]]; then
  compose_args+=(-f "$PROJECT_DIR/.docker-dev/compose.user.yaml")
fi

if [[ -s "$SERVICES_FILE" ]]; then
  compose_args+=(-f "$SERVICES_FILE")
fi

while IFS= read -r profile; do
  [[ -n "$profile" ]] && compose_args+=(--profile "$profile")
done < "$PROFILES_FILE"

docker compose "${compose_args[@]}" config --quiet
docker compose "${compose_args[@]}" up -d
```

Scelta Perl

La struttura è identica; cambia solo l'invocazione:

```
PERL_BIN="${PERL_BIN:-perl}"
SERVICES_SCRIPT="${DOCKER_DEV_ROOT}/bin/docker-dev-services.pl"

"$PERL_BIN" "$SERVICES_SCRIPT" \
  --target "$TARGET" \
  --project-name "$PROJECT_NAME" \
  --with "$DDEV_WITH" \
  --services-file "$SERVICES_FILE" \
  --profiles-file "$PROFILES_FILE" \
  ${DDEV_BUILD_MISSING:+--build-missing} \
  ${DDEV_DRY_RUN:+--dry-run}
```

Il resto di `docker-dev.sh` — costruzione dell'array `compose_args`, lettura profili, `config --quiet`, `up -d` — non cambia.

Raccomandazione finale

Userei **Python + PyYAML + Jinja2**, con questi vincoli:

- `service.yaml` viene letto con `yaml.safe_load`, mai `yaml.load` non vincolato.[1] [4]
- I template Jinja2 usano `StrictUndefined`, così una variabile mancante fallisce invece di produrre YAML incompleto.
- `40.services.sh` può rimanere un sottile wrapper Bash che chiama `bin/docker-dev-services.py`; così la CLI di `docker-dev` resta coerente.
- Il file generato è `generated/compose.services.yaml`, non `compose.override.yaml`.
- Ogni servizio possiede il proprio `compose.yaml.j2`; lo script Python si limita a risolvere, validare, renderizzare e comporre.
- Prima di `up`, esegui sempre `docker compose ... config --quiet`: è il controllo più economico per intercettare YAML o merge non validi.

Perl + Template::Toolkit sarebbe una scelta eccellente se volessi mantenere `docker-dev` interamente in un ecosistema Perl/Raku o se i template dovessero diventare molto sofisticati. Per un tool DevOps locale, però, Python riduce attrito, dipendenze e superficie di manutenzione.[3] [2]

⁂

1. https://pyyaml.org/wiki/PyYAMLDocumentation
2. https://tt2.org/docs/manual/Intro.html
3. https://template-toolkit.org/docs/index.html
4. https://github.com/yaml/pyyaml
5. https://docs.docker.com/compose/how-tos/multiple-compose-files/include.md
6. https://template.mailkit.com/manual/Variables.php

7. https://www.geeksforgeeks.org/python/parse-a-yaml-file-in-python/
8. https://nhimg.org/articles/pyyaml-api-safety-depends-on-version-and-loader-choice/
9. https://stackoverflow.com/questions/1773805/how-can-i-parse-a-yaml-file-in-python
10. https://docs.docker.com/reference/compose-file/merge/
11. https://docs.docker.com/compose/
12. https://docs.docker.com/reference/compose-file/
13. https://earthly.dev/blog/yaml-in-python/
14. https://realpython.com/python-yaml/
15. https://www.template-toolkit.org/docs/manual/Directives.html